

Koopman MPC for Lateral Vehicle Control in the comma.ai Controls Challenge

Noah Miller

ME8930: Data-Driven Learning and Control of Dynamic Systems
School of Mechanical and Automotive Engineering,
Clemson University, Greenville, SC 29607 USA
Email: nhmille@g.clemson.edu

Abstract—This report studies data-driven model predictive control (MPC) for lateral vehicle control on the comma.ai Controls Challenge. The controller must command steering at 10 Hz to accurately track a desired lateral acceleration while avoiding excessive jerk. Two learned predictive models are compared inside the same constrained MPC formulation: a linear autoregressive predictor and a finite-dimensional Koopman/EDMD predictor with quadratic observables. Both models are identified from closed-loop PID rollouts, while model validation and closed-loop evaluation are performed on route segments separate from identification. The learned models are accurate one-step predictors of the PID-collected dynamics, and recursive rollout tests show only a small but consistent advantage for the Koopman model. In closed-loop evaluation on 999 evaluation-reserved routes, both MPC controllers substantially improve over the PID baseline, and the Koopman MPC gives a small but persistent improvement over the linear MPC under identical weights and constraints. The final challenge-oriented configuration achieved a submitted score of 74.287, placing the controller in the middle of the leaderboard. The results suggest that the main practical value of the Koopman lift in this setting is not a dramatic one-step prediction improvement, but a modest reduction in recursive model error that can translate into better receding-horizon tracking.

Index Terms—Koopman operator, model predictive control, EDMD, autonomous vehicle control, lateral acceleration tracking, data-driven control.

I. INTRODUCTION

Autonomous driving is often lauded as a canonical controls challenge, requiring perception, prediction, and planning in unstructured environments with plentiful disturbances and demanding response times. Even if the higher-level system is competent, its decisions are executed by low-level actuators with limited bandwidth, modeling error, noise, and safety constraints. In lateral vehicle control, steering must precisely track a desired target while remaining robust in varying operating conditions such as bank, roll, and speed. Assuming a high-quality trajectory, lateral control still must be balanced as tracking aggressively can reduce lateral-acceleration error but induce excessive jerk, thereby ruining consumer comfort or producing unstable behavior, whereas an overly conservative controller may be sluggish and accumulate tracking error. This tradeoff between precision and comfort is a core difficulty in autonomous driving and controls.

The comma.ai Controls Challenge isolates this low-level lateral-control problem by providing route segments, vehicle-

context measurements, target lateral acceleration, and a simulator for closed-loop evaluation [1]. In this setting, the controller receives the current vehicle state and short preview information, then outputs steering commands at 10 Hz with the goal of tracking the desired lateral acceleration without excessive jerk. This benchmark is therefore well suited for studying model predictive control, since MPC provides a structured way to balance tracking accuracy, actuator smoothness, and constraint handling within a common optimization framework.

Recent progress in artificial intelligence has made data-driven methods increasingly attractive for such problems. Learned components can capture nonlinear effects and adapt to operating conditions, but they are often difficult to interpret and can require substantial data for training. As a result, many practical systems favor structured uses of learning rather than fully end-to-end control. This distinction is relevant even in comma.ai’s openpilot, where discussion of low-level control continues to emphasize the challenge of obtaining reliable steering behavior in purely data-driven approaches despite a strong overall autonomy stack [2].



Fig. 1. Comma 3X autonomous driving add-on. Image source: comma ai

Therefore, we focus on a structured learning approach as opposed to an end-to-end learned steering policy. The central question is whether a learned lifted predictor can improve MPC tracking over a simpler linear autoregressive model when both controllers use the same objective, constraints, preview

information, and evaluation protocol. MPC is attractive for this benchmark because it provides explicit weights to balance tracking against steering magnitude and steering-rate regularization while enforcing actuator constraints over a finite horizon and thus depends strongly on the predictive model.

The final implementation compares three controllers: the provided PID baseline, a linear-model MPC, and a Koopman MPC. The two MPC controllers differ only in the prediction model. The linear model uses lagged lateral acceleration, lagged steering, current steering, and measured context variables directly. The Koopman model uses the same control and context variables, but lifts the measured lateral-acceleration history through a quadratic observable dictionary before fitting linear dynamics in the lifted space. The comparison therefore targets a narrow but useful question: does the additional lifted representation provide enough predictive benefit to justify the added complexity for this benchmark?

Evaluation is handled through route-level separation between identification, validation, and closed-loop testing. The predictive models are trained from PID-collected closed-loop rollouts, but the main comparison between linear and lifted models is performed on route segments not included in the identification matrices. This split is important because adjacent transition rows from the same route are highly correlated; holding out random rows would overstate generalization. The final results indicate that the linear model is an unexpectedly strong baseline, while Koopman MPC gives a small but consistent improvement in recursive prediction and closed-loop cost.

II. RELATED WORK

Learning-based control for autonomous driving spans a continuum from end-to-end learned policies to structured architectures in which learning is inserted only where it improves a conventional control stack. End-to-end driving approaches map sensory inputs or compact scene representations directly to steering commands [3], while conditional imitation learning uses high-level commands to guide the learned policy [4], [5]. Other work argues for a more conservative use of learning, where learned quantities support a traditional controller rather than outright replacing the control policy [6]. The present project follows this structured viewpoint; state and context information are supplied by the benchmark, so the learning problem is not full driving-policy synthesis, but rather predictive modeling for low-level control.

Model predictive control is attractive for low-level vehicle control because it optimizes tracking performance over a finite horizon while explicitly handling actuator and state constraints. This receding-horizon structure is a central feature of constrained MPC [7], [8]. In vehicle applications, MPC has been used for active steering and trajectory tracking due to its ability to explicitly manage tracking accuracy, control effort, and constraint satisfaction in a single optimization problem [9]. Its practical performance, however, depends strongly on the prediction model used inside the optimizer. This has motivated learning-enhanced MPC methods for autonomous driving and

tracking, including residual and error-dynamics models that improve prediction accuracy while retaining the structure of MPC [10]–[12].

Koopman-based modeling provides a complementary approach to learning predictive models for control. The Koopman operator gives a linear, infinite-dimensional representation of nonlinear dynamics through the evolution of observable functions. Extended Dynamic Mode Decomposition (EDMD) approximates this operator from snapshot pairs using a finite dictionary of observables [13]. For actuated systems, related methods such as Dynamic Mode Decomposition with control separate autonomous dynamics from the effect of control inputs [14]. Korda and Mezić extended this idea directly to controlled systems and MPC by constructing linear predictors in a lifted observable space, allowing nonlinear dynamics to be approximated in a form compatible with linear predictive control [15].

This compatibility makes Koopman-based control appealing, but it also brings its own practical limitations. The choice of observable dictionary is highly problem-dependent, finite-dimensional Koopman approximations are not guaranteed to capture all relevant nonlinear behavior, and including the original state in a finite invariant subspace can be restrictive [16]. These limitations are especially relevant in vehicle-control settings, where data is noisy, operating conditions vary, and the controller may visit states that are poorly represented in the identification data. Recent surveys and vehicle applications show that Koopman and deep Koopman models can be useful for vehicle dynamics and trajectory tracking, but also reinforce the importance of dictionary choice, data coverage, and robustness [17], [18].

These collective works motivate the comparison in this report. Rather than replacing the entire controller with a learned policy, both learned models are embedded in the same constrained MPC formulation. The linear autoregressive model provides a strong low-complexity baseline, while the Koopman/EDMD model tests whether a simple quadratic lift provides additional predictive value. Recent residual Koopman MPC work suggests that lifting may be most effective not as a complete replacement for a strong nominal model, but as a structured correction channel [19]. The present study therefore evaluates whether a full lifted predictor provides measurable benefit over a linear predictor on the comma.ai Controls Challenge, under matched objectives, constraints, preview information, and unseen route-level evaluation.

III. METHODOLOGY

A. Challenge Benchmark and Notation

The comma.ai Controls Challenge provides route segments and an autoregressive simulator, *TinyPhysics*, for lateral acceleration. At time step k , the route data includes ego speed $v_{\text{ego},k}$, longitudinal acceleration $a_{\text{ego},k}$, road roll φ_k , desired lateral acceleration y_k^{ref} , and a logged steering command u_k^{log} . During simulator warmup, the logged command is applied to initialize the autoregressive history. After control begins,

the steering command from the controller u_k is used and the simulator returns the realized lateral acceleration y_k .

We distinguish between three types of variables: the controlled state x_k , the chosen input u_k , and external measurements d_k which are defined

$$d_k = [\varphi_k \quad v_{\text{ego},k} \quad a_{\text{ego},k}]^\top. \quad (1)$$

Along the prediction horizon, the controller is then provided preview sequences

$$d_{k+i}, \quad y_{k+i}^{\text{ref}}, \quad i = 0, \dots, N-1.$$

The challenge cost has two components: lateral-acceleration tracking cost and jerk cost. The total evaluation cost is

$$J_{\text{eval}} = 50J_{\text{lat}} + J_{\text{jerk}}. \quad (2)$$

B. Linear Prediction Model

The one-step prediction problem is written as

$$y_{k+1} = f(y_{k-1}, u_{k-1}, y_k, u_k, d_k), \quad d_k = [\varphi_k \quad v_k \quad a_k]^\top$$

and the linear model is an affine one-step predictor for lateral acceleration. The regression vector is

$$\phi_k = [y_{k-1} \quad u_{k-1} \quad y_k \quad u_k \quad \varphi_k \quad v_k \quad a_k]^\top, \quad (3)$$

with affine model

$$y_{k+1} \approx \theta^\top \phi_k + c. \quad (4)$$

Given training pairs $\{(\phi_k, y_{k+1})\}_{k=0}^{M-1}$, we define the stacked data matrices

$$\Phi = \begin{bmatrix} \phi_0^\top \\ \phi_1^\top \\ \vdots \\ \phi_{M-1}^\top \end{bmatrix}, \quad Y^+ = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix}. \quad (5)$$

The affine constant is handled by appending a column of ones

$$\bar{\Phi} = [\Phi \quad \mathbf{1}], \quad \bar{\theta} = \begin{bmatrix} \theta \\ c \end{bmatrix}. \quad (6)$$

The least-squares problem for the parameters is then

$$\hat{\theta} = \arg \min_{\theta} \|\bar{\Phi}\bar{\theta} - Y^+\|_2^2. \quad (7)$$

For use in MPC, this predictor is rewritten as an augmented linear state-space model

$$x_k^{\text{lin}} = [y_k \quad y_{k-1} \quad u_{k-1} \quad 1]^\top \quad (8)$$

which yields an affine linear system

$$x_{k+1}^{\text{lin}} = A_{\text{lin}} x_k^{\text{lin}} + B_{\text{lin}} u_k + E_{\text{lin}} d_k, \quad y_k = C_{\text{lin}} x_k^{\text{lin}}. \quad (9)$$

C. Koopman/EDMD Prediction Model

The Koopman model uses the measured history state

$$s_k = [y_{k-1} \quad y_k]^\top. \quad (10)$$

A quadratic observable dictionary is applied:

$$z_k = \Psi(s_k) = [y_{k-1} \quad y_k \quad y_{k-1}^2 \quad y_k^2 \quad y_{k-1}y_k \quad 1]^\top. \quad (11)$$

The control and context variables are not lifted. Instead, they enter linearly through

$$w_k = [u_{k-1} \quad u_k \quad \varphi_k \quad v_k \quad a_k]^\top. \quad (12)$$

The lifted dynamics are modeled as

$$z_{k+1} = A_K z_k + B_K w_k \quad (13)$$

where w_k includes the exogenous context. Because the raw state coordinates are retained in z_k , lateral acceleration is recovered by the linear output map

$$y_k = H_K z_k, \quad H_K = [0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0]. \quad (14)$$

Given the lifted data matrices

$$Z = [z_0 \quad z_1 \quad \dots \quad z_{M-1}], \quad Z^+ = [z_1 \quad z_2 \quad \dots \quad z_M], \quad (15)$$

and

$$W = [w_0 \quad w_1 \quad \dots \quad w_{M-1}], \quad (16)$$

we solve the EDMD least-squares problem as

$$(\hat{A}_K, \hat{B}_K) = \arg \min_{A_K, B_K} \|Z^+ - A_K Z - B_K W\|_F^2. \quad (17)$$

This yields a finite-dimensional approximation of the Koopman in which the nonlinearity is captured through the observable map $\Psi(\cdot)$, while the applied input and measured context enter linearly. The result is a linear-MPC compatible structure.

D. MPC Formulation

At each time step k , the controller solves for a horizon of steering commands

$$U_k = [\hat{u}_{k|k} \quad \hat{u}_{k+1|k} \quad \dots \quad \hat{u}_{k+N-1|k}]^\top. \quad (18)$$

The predicted state trajectory is generated by either the linear model in (9) or the Koopman model in (13). The same cost, input bounds, input-rate bounds, and preview sequences are used for both controllers.

The finite-horizon objective is

$$J(U_k) = \sum_{i=0}^{N-1} \left[Q_y (\hat{y}_{k+i+1|k} - y_{k+i+1}^{\text{ref}})^2 + R_u \hat{u}_{k+i|k}^2 + R_{\Delta u} (\hat{u}_{k+i|k} - \hat{u}_{k+i-1|k})^2 \right] + Q_f (\hat{y}_{k+N|k} - y_{k+N}^{\text{ref}})^2. \quad (19)$$

This is subject to actuator magnitude and rate constraints,

$$u_{\min} \leq \hat{u}_{k+i|k} \leq u_{\max}, \quad i = 0, \dots, N-1, \quad (20)$$

$$|\hat{u}_{k+i|k} - \hat{u}_{k+i-1|k}| \leq \Delta u_{\max}, \quad i = 0, \dots, N-1. \quad (21)$$

After the optimization is solved, only the first command $\hat{u}_{k|k}$ is applied, the simulator advances by one step, and the problem is solved again with updated measurements and preview.

The final weight set used for the main closed-loop comparison was

$$Q_y = 15, \quad Q_f = 2, \quad R_u = 0, \quad R_{\Delta u} = 500. \quad (22)$$

The horizon was varied for the ablation study. Unless otherwise stated, the training-size comparison used a representative horizon of $N = 50$.

E. Data Collection and Evaluation Split

Data for model identification was collected by running the provided PID controller in closed-loop simulation, thus producing stable rollouts from which one-step transition pairs could be constructed. For each route, rows were formed using the current and prior lateral acceleration, steering, and measured context variables. We stress that data was split at the route-segment level rather than at the individual-row level, so validation and evaluation trajectories were not partially duplicated in the training matrices.

Two training sets were used: the smaller set contained 1,000 PID rollouts, corresponding to approximately 4.97×10^5 transition rows, split into 800 training segments and 200 validation segments while the larger set contained 10,000 PID rollouts, corresponding to approximately 4.97×10^6 transition rows, split into 8,000 training segments and 2,000 validation segments. The closed-loop results reported in the main tables use a separate set of 999 route segments unused for training. This is distinct from the final challenge-style evaluation, which was run on 5,000 benchmark segments and produced the submitted score.

This separation is useful for two reasons. First, it prevents optimistic validation from correlated rows in the same route. Second, it tests whether the learned model and MPC tuning generalize to the 999 route segments that were not used to fit the dynamics. The closed-loop evaluation is therefore a stricter test than simply replaying training routes with the MPC controller.

IV. RESULTS

The evaluation is organized into four parts. First, the learned predictors are tested on one-step validation pairs and recursive rollout prediction to determine whether the Koopman lift meaningfully improves model accuracy. Second, the PID baseline, linear MPC, and Koopman MPC are compared in closed-loop simulation on 999 route segments that are separate from the identification/training set. Third, the prediction horizon is varied to study sensitivity to look-ahead length and gauge the capabilities of the prediction models. Finally, representative trajectories and stress tests are used to interpret how the controllers behave beyond their aggregate costs.

TABLE I
ONE-STEP PREDICTION VALIDATION ON UNSEEN TRANSITION PAIRS.

Model	Training	RMSE	MAE	R^2
Linear	1000	0.040085	0.028168	0.993121
Koopman	1000	0.040118	0.028149	0.993109
Linear	10000	0.040090	0.028159	0.993119
Koopman	10000	0.040086	0.028151	0.993120
Persistence	—	0.050530	0.033268	0.989068

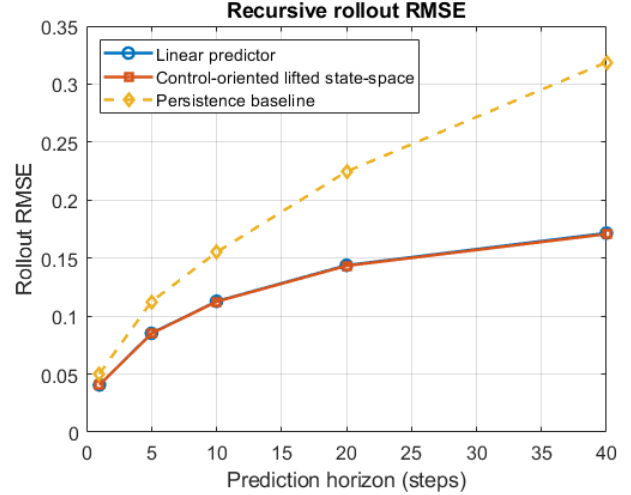


Fig. 2. Recursive rollout prediction error. One-step metrics are nearly identical, but the Koopman/EDMD predictors show a small advantage when the model is rolled forward over longer horizons.

A. Prediction Validation

Table I summarizes the one-step validation results for the 1,000- and 10,000-rollout trained linear and Koopman prediction models. The persistence baseline predicts $\hat{y}_{k+1} = y_k$. Both learned models outperform persistence by a wide margin, but the linear and Koopman predictors are nearly indistinguishable at one step. The most important observation from Table I is that the linear autoregressive model is already a strong local predictor. Once prior lateral acceleration, prior steering, current steering, and measured context are included, the PID-visited dynamics appear to be well approximated by a linear map for one-step prediction, therefore the Koopman lift does not produce a visible one-step advantage.

The recursive rollout results shown in Figure 2 are more control-relevant since the MPC performance depends strongly on the use of the model recursively over a finite horizon. The Koopman models are observed to consistently accumulate slightly less error than their linear counterparts, while both learned models remain well below the persistence baseline over increasing prediction horizons. The margin is small, but its consistency suggests that the lift is not improving instantaneous prediction (as indicated in the one-step evaluation in Table I) so much as reducing degradation under repeated model application.

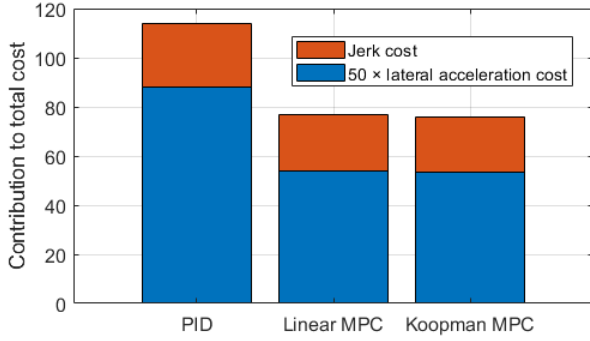


Fig. 3. Cost decomposition for the 10,000-rollout MPC controllers on the evaluation set.

B. Closed-Loop Evaluation

Figure 3 and Table II summarize the closed-loop results on the 999-route evaluation set. Both MPC controllers handily outperform the provided PID baseline. For the 10,000-rollout models, the linear MPC reduces total cost from 114.083 to 76.732, while the Koopman MPC further reduces it to 76.115. Thus, most of the performance gain comes from replacing the PID controller with MPC, while the Koopman lift provides a smaller additional improvement over the linear MPC.

The cost decomposition shown in Figure 3 indicates that the improvement by Koopman MPC is primarily through lateral-acceleration tracking while the jerk terms for the linear and Koopman MPC controllers are nearly identical, which suggests that the Koopman controller is not improving the score by simply accepting rougher behavior. Instead, the lifted model allows slightly better tracking without incurring additional jerk.

We also present the results for the 1,000-run-trained MPC controllers in Table II to consider the effect of training-set size; the effect is present but modest. We note that the increase from 1,000 to 10,000 rollouts improves the Koopman total cost from 76.527 to 76.115 while the corresponding linear-model change is smaller. This indicates the simple model class may be near saturation for the chosen regressors, while the lifted model may have slightly more capacity to benefit from additional data.

TABLE II
CLOSED-LOOP EVALUATION RESULTS.

Training	Controller	Lat.	Jerk	Total
-	Baseline PID	1.759	26.110	114.083
1000	Linear MPC	1.083	22.847	76.993
1000	Koopman MPC	1.074	22.820	76.527
10000	Linear MPC	1.081	22.661	76.732
10000	Koopman MPC	1.069	22.642	76.115

C. Horizon Ablation

Table III and Figure 4 summarize the horizon sweep for the 10,000-rollout models. Horizon length has a visible but non-monotonic effect on closed-loop performance. The Koopman

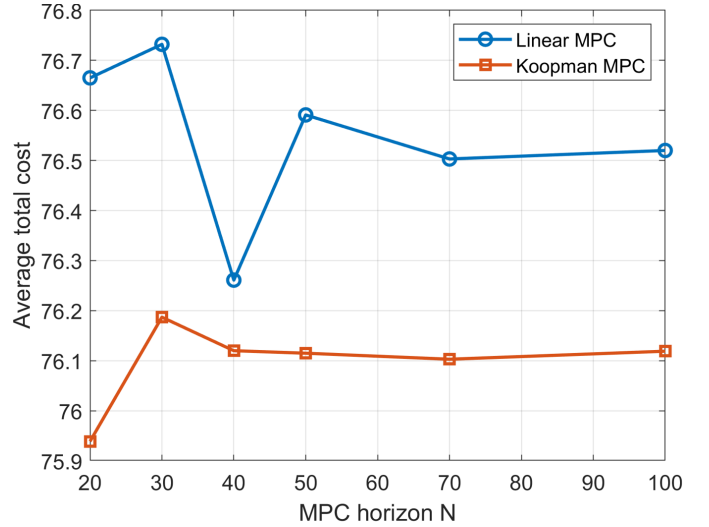


Fig. 4. Closed-loop horizon ablation on evaluation routes. The Koopman MPC is lower at every tested horizon, and both controllers plateau for longer horizons.

MPC has lower average cost than the linear MPC at every tested horizon, with differences ranging from 0.141 to 0.726. These are small in absolute terms, but their consistency supports the conclusion that the lifted predictor is beneficial when embedded in MPC.

For this benchmark, increasing the horizon beyond approximately $N = 40$ did not produce a clear additional benefit. The observed plateau is intuitive as longer horizons will only help when the model remains accurate that far ahead even in the presence of accumulated error. Eventually, prediction mismatch and target-signal fluctuations will limit the value of additional lookahead, tapering the benefit. A horizon of $N = 50$ was retained for the main training-size comparison because it lies in this plateau region and represents a long-horizon setting, even though the lowest Koopman cost in this sweep occurred at $N = 20$.

D. Trajectory and Tuning Behavior

The aggregate cost differences are also visible in representative trajectories. Figure 5 compares a sample segment trajectory of the Koopman MPC and compares against the PID baseline in the top panel and the Linear MPC on the bottom. The PID baseline can lag the desired lateral acceleration during more active portions of the route, while the MPC response is

TABLE III
HORIZON ABLATION ON EVALUATION ROUTES. GAIN IS LINEAR MPC TOTAL COST MINUS KOOPMAN MPC TOTAL COST.

N	Linear MPC	Koopman MPC	Gain
20	76.665	75.939	0.726
30	76.732	76.187	0.545
40	76.261	76.120	0.141
50	76.591	76.115	0.476
70	76.503	76.103	0.400
100	76.520	76.119	0.401

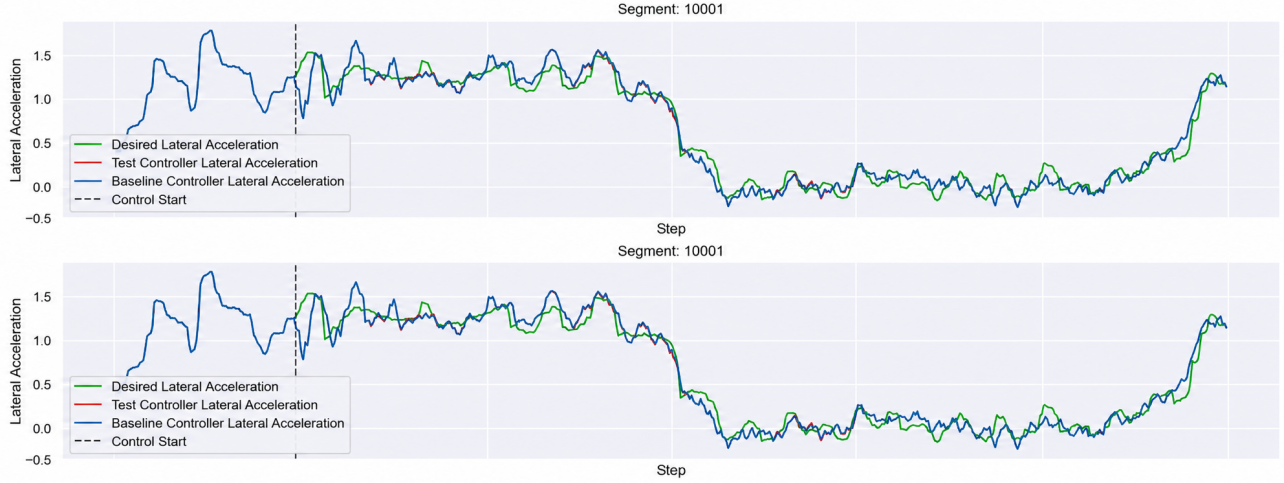


Fig. 5. Representative trajectory comparison on segment 10001. The Koopman MPC tracks the desired lateral acceleration more closely than the PID baseline, while the Koopman and linear MPC trajectories remain visually close.

closer to the target. The linear and Koopman MPC trajectories are visually very close, which agrees with the small average-cost gap between the two learned controllers.

The "stress-test" trajectories in Figure 6 aim to highlight why some tuning observations and why smoothness regularization is essential for this benchmark. Increasing the tracking weight alone produced rougher behavior but did not immediately destabilize the controller on the shown route; it still outperformed the PID baseline on that segment (104.799 vs. 139.010). However, reducing the rate penalty as well caused large oscillations in the realized lateral acceleration. This behavior suggests that the target lateral-acceleration traces *can* destabilize the MPC controller should steering-rate regularization is underweighted.

We also note that in the final tuning, the steering magnitude was not penalized; we found better performance by only penalizing the steering-rate due to control effort not being a scored aspect, only jerk. Therefore, the steering-rate penalty acts not only as a comfort term, but also as the only regularizer against chasing noisy target fluctuations while the constraint is reserved for a safety guard.

E. Summary and Limitations

The results support the original hypothesis, but with a notable qualification: Koopman MPC consistently outperforms the linear MPC in recursive prediction and closed-loop evaluation, but the improvement is quite modest. Consequently, the dominant finding is that the linear autoregressive model is a much stronger baseline than expected once the measured context, steering history, and lateral-acceleration history are included.

The modest benefit from increasing the training set from 1,000 to 10,000 PID rollouts suggests that the chosen models may be more limiting than the amount of data. A richer observable dictionary, additional delay embedding, or explicit disturbance modeling may provide more benefit than simply

adding more PID-collected transition rows to the same regression structure. We note that while applying the prediction models captured general trends during rollout, quick responses tended to wash out increasingly with longer horizons.

A remaining limitation is that both learned models are identified from PID-visited dynamics. Although the route-level split prevents optimistic validation from correlated rows within the same route, the MPC controllers may struggle to respond well should the state move partially outside the PID data distribution. The input and rate constraints help limit this distribution shift, but do not eliminate it. This makes residual Koopman MPC a natural future direction: the strong linear MPC could provide the nominal behavior, while a lifted residual model supplies bounded corrections for systematic nonlinear errors.

A final limitation is that the simulator itself is autoregressive rather than a direct physical ground truth. The initial warmup period initializes the simulator history before control is applied, so the effective prediction context can vary from route to route. We hypothesize this could partially smooth or filter the dynamics observed in PID-generated closed-loop rollouts relative to what might be obtained from open-loop responses. Although the raw datasets include logged state progression and steering inputs, the reported models were more reliable when identified from closed-loop simulator rollouts.

V. CONCLUSION AND FUTURE WORK

This project implemented and evaluated linear MPC and Koopman MPC for the comma.ai Controls Challenge. Both predictive models were trained from PID closed-loop rollouts and evaluated on route segments separated from identification. The linear model achieved nearly the same one-step prediction accuracy as the Koopman/EDMD model, showing that a carefully chosen autoregressive predictor is already a strong baseline for this benchmark.

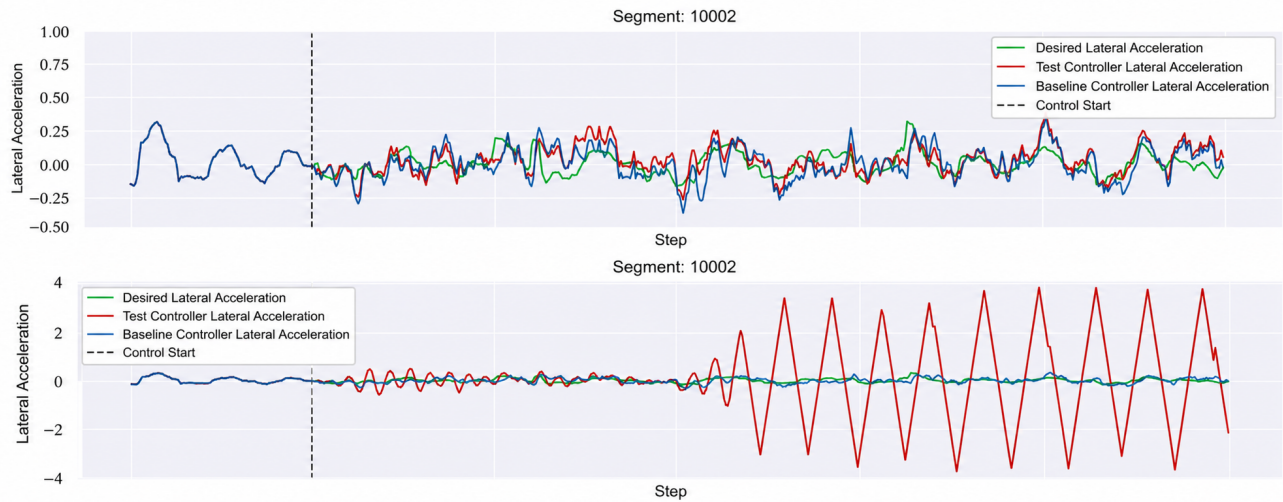


Fig. 6. Stress-test behavior on segment 10002. The upper plot shows a high-tracking-weight case that remains stable but rougher. The lower plot shows that reducing the rate penalty can induce large oscillations.

Nevertheless, on the 999-route evaluation, Koopman MPC produced a small but consistent improvement over linear MPC under equal weights and constraints, primarily through lower lateral-acceleration tracking cost while maintaining similar jerk cost. The horizon ablation showed that Koopman MPC remained lower at all tested horizons and that performance plateaued for horizons beyond roughly $N = 40$. The final challenge-oriented run achieved a submitted score of 74.287.

Overall, the results support using Koopman lifting as a structured, MPC-compatible modeling improvement, but not as a dramatically better replacement for a well-designed linear autoregressive model on this benchmark. Future work will focus on alternative controller formulations such as residual Koopman correction, noise-aware identification, disturbance-aware lifting, and broader dictionary comparisons to further improve the Koopman implementation.

ACKNOWLEDGMENT

This report was prepared as the final project for ME8930: Data-Driven Learning and Control of Dynamic Systems.

REFERENCES

- [1] comma.ai, “comma Controls Challenge v2,” GitHub repository, 2026. [Online]. Available: https://github.com/commaai/controls_challenge
- [2] “RL for car controls,” comma.ai blog, Sep. 2024. [Online]. Available: <https://blog.comma.ai/rlcontrols/>
- [3] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, “End to end learning for self-driving cars,” 2016.
- [4] F. Codevilla, M. Muller, A. Lopez, V. Koltun, and A. Dosovitskiy, “End-to-end driving via conditional imitation learning,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 4693–4700.
- [5] J. Hawke, R. Shen, C. Gurau, S. Sharma, D. Reda, N. Nikolov, P. Mazur, S. Micklethwaite, N. Griffiths, A. Shah, and A. Kendall, “Urban driving with conditional imitation learning,” 2020.
- [6] C. Chen, A. Seff, A. Kornhauser, and J. Xiao, “DeepDriving: Learning affordance for direct perception in autonomous driving,” 2015.
- [7] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, “Constrained model predictive control: Stability and optimality,” *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.
- [8] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.
- [9] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, “Predictive active steering control for autonomous vehicle systems,” *IEEE Transactions on Control Systems Technology*, vol. 15, no. 3, pp. 566–580, 2007.
- [10] A. Bertipaglia, M. Alirezai, R. Happee, and B. Shyrokau, “A learning-based model predictive contouring control for vehicle evasive manoeuvres,” 2024.
- [11] H. Xue, M. Brown, and J. C. Gerdes, “Learning model predictive control with error dynamics regression for autonomous racing at the limit of handling,” 2023.
- [12] Y. Li, C. Huang, D. Yang, W. Liu, and J. Li, “Learning based mpc for autonomous driving using a low dimensional residual model,” 2024.
- [13] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, “A data-driven approximation of the koopman operator: Extending dynamic mode decomposition,” *Journal of Nonlinear Science*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [14] J. L. Proctor, S. L. Brunton, and J. N. Kutz, “Dynamic mode decomposition with control,” *SIAM Journal on Applied Dynamical Systems*, vol. 15, no. 1, pp. 142–161, 2016.
- [15] M. Korda and I. Mezić, “Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control,” *Automatica*, vol. 93, pp. 149–160, 2018.
- [16] S. L. Brunton, B. W. Brunton, J. L. Proctor, and J. N. Kutz, “Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control,” *PLOS ONE*, vol. 11, no. 2, p. e0150171, 2016.
- [17] W. Manzoor, S. Rawashdeh, and A. Mohammadi, “Vehicular applications of koopman operator theory – a survey,” *IEEE Access*, vol. 11, pp. 25 917–25 931, 2023.
- [18] Y. Xiao, X. Zhang, X. Xu, X. Liu, and J. Liu, “Deep neural networks with koopman operators for modeling and control of autonomous vehicles,” *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 1, pp. 135–146, 2023.
- [19] Y. Fu, C. Hu, H. Xiong, Z. Bao, W. Du, E. Ghignone, M. Magno, L. Xie, and H. Su, “Residual koopman model predictive control for enhanced vehicle dynamics with small on-track data input,” 2025.